

Library Management System

Re-engineering a coursework web application into a documented,
verifiable reference implementation

PROJECT REPORT

Prepared by	Piyush Anand
Roll number	CS21B1019
Domain	Web application engineering, database design, software verification
Implementation	PHP 7.4+, MySQL 5.7+ / MariaDB 10.3+, Bootstrap 3
Date	22 September 2026

*This report documents the design, implementation and verification of a library
circulation system, and states plainly which claims in it were executed and
which were not.*

Contents

1. Introduction	4
1.1 Context and motivation	4
1.2 Objectives	4
1.3 Scope	4
1.4 Summary of outcomes	4
2. System overview	5
2.1 Member portal	5
2.2 Librarian panel	5
3. Architecture	6
3.1 Why the rules are centralised	6
3.2 Database access settings	6
3.3 Technology choices	7
4. Data model	8
4.1 Tables	8
4.2 Two decisions worth defending	9
5. Circulation logic	10
5.1 The policy	10
5.2 Fine arithmetic	10
5.3 Issuing a copy	11
5.4 Receiving a copy	12
6. Security engineering	14
6.1 Threats and controls	14
6.2 Migrating passwords without a reset	14
6.3 Residual risk	15
7. Verification and validation	16
7.1 Static analysis	16
7.2 Unit tests of the database-free logic	16
7.3 A stress harness without PHP	17
7.4 Card-number allocation	18
7.5 Concurrency experiments	18
7.6 Threats to the validity of these results	18
8. Defects found and resolved	20

9. Results and discussion	20
9.1 What the evidence supports	20
9.2 What the evidence does not support	21
9.3 Reflection on method	21
10. Limitations and future work	22
11. Conclusion	22
Appendix A. Installing and running	23
Appendix B. Repository layout	23
Appendix C. Reproducing the results in this report	23

1. Introduction

1.1 Context and motivation

A small academic library performs a handful of operations over and over: a member looks for a title, a librarian hands over a copy, the copy comes back, and a fine is charged when it comes back late. The operations are simple to describe, which makes it easy to build software for them that appears to work and is quietly wrong: a copy count that drifts, a fine computed by hand, a catalogue that anyone who is logged in can delete.

This project began as a coursework web application in PHP and MySQL. It had the expected shape of such a project: server-rendered pages, a Bootstrap front end, one file per screen. It also had the expected defects. This report documents its re-engineering into a reference implementation that is documented, constrained by its schema, and verified as far as the available tooling allowed.

1.2 Objectives

- **Make the system installable.** The database schema was absent from the repository, so the application could not be deployed at all by anyone who did not already possess the database.
- **Give the circulation rules one home.** Loan period, fine rate and borrowing limit should be parameters of the system, not constants scattered through page scripts.
- **Make the circulation operations correct under concurrency.** Issuing and returning a copy each touch three tables; they must be atomic and must not lose or invent copies.
- **Bring the security posture to a deployable standard** for passwords, SQL, output encoding, request forgery and privilege separation.
- **Verify the result**, and report honestly on what the verification does and does not cover.

1.3 Scope

The work deliberately preserves the application's external shape. The framework-free PHP, the page-per-URL structure, the Bootstrap 3 front end and the original table names — including the misspelled column `RetrunStatus` — are unchanged. Renaming these would have produced a different project rather than a corrected one, and would have made the result impossible to compare against the version it grew from. The changes are concentrated in the data model, the domain logic, the security layer and the documentation.

1.4 Summary of outcomes

Area	Outcome
Data model	7 tables with primary, unique and foreign keys, InnoDB and utf8mb4, delivered as an idempotent schema script with seed data
Domain layer	Circulation rules, fine arithmetic and transactional issue/return isolated in <code>includes/library.php</code> ; policy expressed as three constants
Security	bcrypt with transparent migration from md5, CSRF tokens on every state-changing form, prepared statements throughout, POST-only state changes, session hardening
Defects	9 defects found and fixed, including a privilege-escalation flaw, a copy-count leak and a time-of-check/time-of-use race
Verification	3 static analyses, 17 unit assertions, a 20,000-operation randomised stress run against 6 invariants, and 2 concurrency experiments
Size	41 PHP files, ~4,351 lines; the rework touched 53 files (+5,899 / -3,791 lines)

Table 1. Summary of what the re-engineering produced.

2. System overview

The system serves two roles that never share a screen. A **member** registers, searches the catalogue, sees their own loans and fines, and may request a title the library does not hold. A **librarian** maintains the catalogue, issues and receives copies, tracks overdue loans and fines, blocks or reinstates members, and rules on acquisition requests.

2.1 Member portal

Page	Function
index.php	Login: bcrypt verification, session regeneration, blocked-account handling
signup.php	Self-registration; allocates a library card number under a file lock; validates e-mail, mobile and password; checks e-mail availability over AJAX
dashboard.php	Books borrowed, loans open against the per-member limit, overdue count, fine payable
available-books.php	Catalogue search across title, author and ISBN, with live copy availability
issued-books.php	Personal loan history with due dates, days overdue and the fine accruing
book-requests.php	Raise an acquisition request; see the librarian's decision
my-profile.php	Profile, registration date, outstanding fine balance
change-password.php	Password change with current-password verification

2.2 Librarian panel

Page	Function
dashboard.php	Eight live indicators — titles, copies on shelf, members, blocked members, books out, overdue loans, pending requests, fines outstanding — each linking to the corresponding filtered list
issue-book.php	Issue a copy; AJAX member and title lookup; every circulation rule checked before the ledger is touched
update-issue-bookdetails.php	Receive a copy; the fine is pre-computed from the policy and may be overridden for a waiver or a disputed date
manage-issued-books.php	The circulation ledger, filterable by all / on loan / overdue
add-book.php, manage-books.php, edit-book.php	Catalogue maintenance; ISBN validation, author de-duplication, deletion refused while copies are out
manage-categories.php, manage-authors.php	Classification and author maintenance
mail-student.php	Every member holding an overdue book, with a one-click reminder that itemises their loans and fines
reg-students.php	Member list; block and reinstate accounts
book-request.php	Approve or decline acquisition requests, filterable by status

3. Architecture

The application has no framework, so its layering is a convention rather than something the runtime enforces. The convention is simple and is stated here so that it can be checked: a page script may read input, call helper functions and render HTML; it may not decide what the library permits. Figure 1 shows the arrangement.

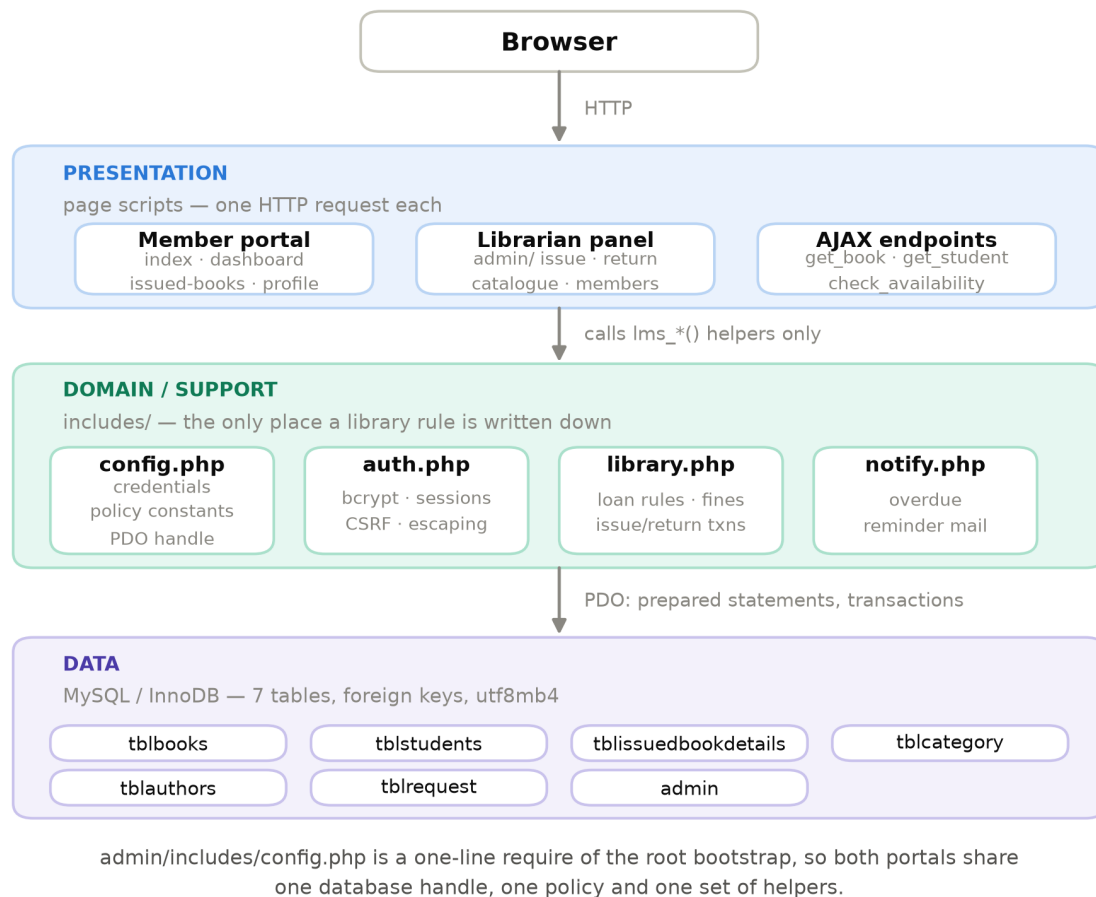


Figure 1. Layered architecture. Page scripts hold presentation only; every library rule lives in the domain layer, and the two portals share a single bootstrap, database handle and policy.

3.1 Why the rules are centralised

Loan period, fine rate and borrowing limit are declared once, in `includes/config.php`, and applied by `includes/library.php`. Changing `LOAN_PERIOD_DAYS` from 14 to 21 changes, in one edit: what the issue screen tells the librarian, the due dates the member sees, the overdue filter on the ledger, the fine pre-filled on return, and who appears on the overdue-reminder list. In the original code the same change would have required finding every page that mentioned a date interval, and the fine was not computed at all — it was typed in by hand.

3.2 Database access settings

The PDO handle is opened with three settings that are worth stating explicitly, because each one rules out a class of defect:

```
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_OBJ
PDO::ATTR_EMULATE_PREPARES => false
```

- **Exceptions on error.** A failed write cannot pass unnoticed. The original code inspected `lastInsertId()` and assumed success otherwise.
- **Native prepares.** Parameters are sent to the server separately from the SQL text, so the placeholder is never interpolated. This also forbids reusing a named placeholder twice in one statement — a constraint that caught two latent errors during this work (Section 8).

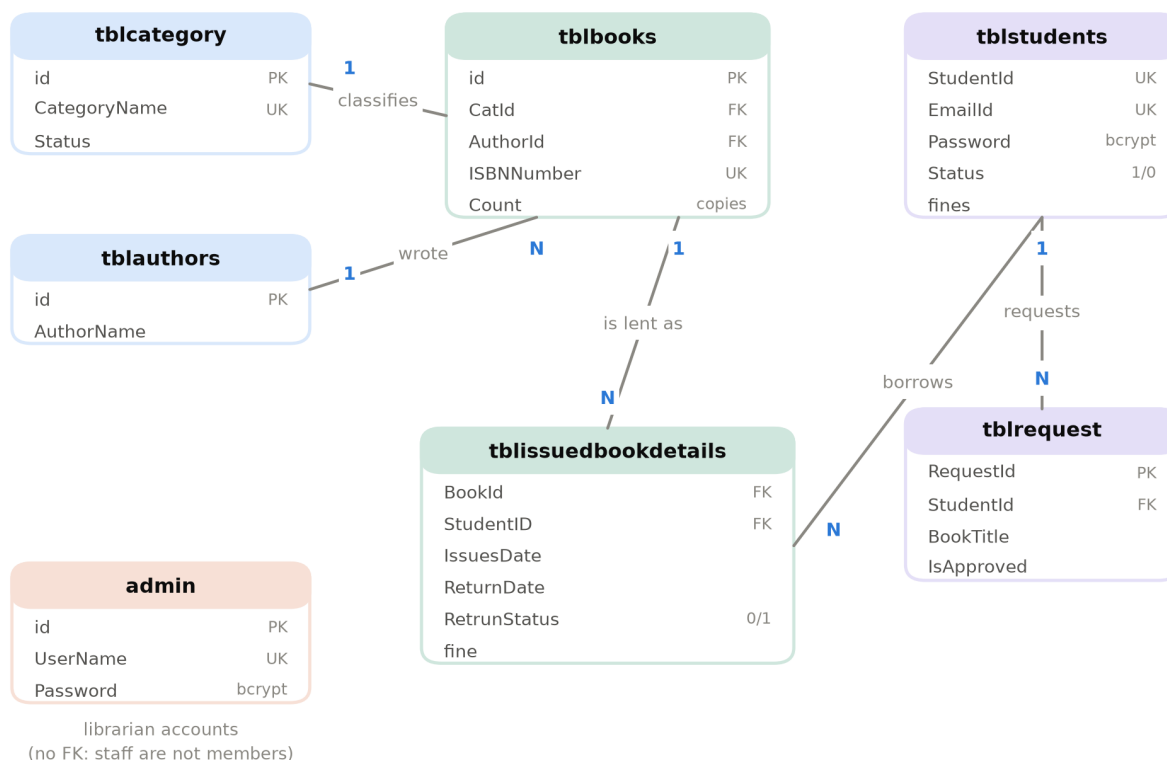
3.3 Technology choices

Layer	Choice	Rationale
Language	PHP 7.4+, no framework	Inherited. Retaining it keeps the project comparable to its origin; the typed helpers and <code>password_*</code> API require 7.x
Database	MySQL 5.7+ / MariaDB 10.3+, InnoDB	Foreign keys and row-level locking are both required by the circulation logic; MyISAM would provide neither
Encoding	utf8mb4	Titles and member names outside the Basic Multilingual Plane are stored correctly
Front end	Bootstrap 3, jQuery 1.10, DataTables	Inherited and deliberately untouched; replacing it was out of scope
Sessions	PHP native sessions, hardened cookies	No external session store is justified at this scale

Table 4. Technology stack and the reason for each choice.

4. Data model

The schema was the largest single gap in the inherited project: it did not exist in the repository. The application could therefore not be installed by anyone who did not already possess a populated database. It is now delivered as `database/Library.sql`, an idempotent script that drops and recreates the schema with seed data, so a reviewer can go from a clone to a running system with one command.



`tblbooks.Count` is the number of copies ON THE SHELF, not the number owned: issuing decrements it, returning restores it, both inside the transaction that writes the ledger row. The ledger joins members by the human-readable card number (`StudentId`) — what a librarian reads off a card at the desk.

Figure 2. Entity-relationship model. Seven tables; foreign keys constrain every reference, and unique keys protect the ISBN, the library card number and the member e-mail.

4.1 Tables

Table	Rows represent	Notable constraints
admin	Librarian accounts	UNIQUE(UserName); Password holds a bcrypt hash
tblcategory	Subject classification	UNIQUE(CategoryName); Status hides a category from drop-downs
tblauthors	Normalised author list	Indexed by name; kept in step with <code>tblbooks</code> by the add/edit flows
tblbooks	Catalogue titles	UNIQUE(ISBNNumber); FK to category and author; Count = copies on the shelf
tblstudents	Members	UNIQUE(StudentId), UNIQUE(EmailId); Status 1 = active, 0 = blocked; fines accumulates
tblissuedbookdetails	One row per issue event	FK to book and to member card number; RetrunStatus 0 = out, 1 = returned; indexed on (RetrunStatus, IssuesDate) for the overdue queries
tblrequest	Acquisition requests	FK to member; IsApproved in {Pending, Approved, Not Approved}

Table 5. The seven tables and the constraints that carry the most design weight.

4.2 Two decisions worth defending

Count is shelf stock, not holdings

`tblbooks.Count` records how many copies are physically available, not how many the library owns. Issuing decrements it; returning restores it. The alternative — storing total holdings and deriving availability by counting open loans — is more normalised, but it makes the availability check a join in the hot path and, more importantly, gives up the single-statement concurrency guard described in Section 5.3. The cost of the chosen design is that the count and the ledger must be written in the same transaction, which is exactly what invariant I2 in Section 7 checks.

Members are keyed twice

A member has a surrogate key (`id`) and a human-readable library card number (`StudentId`, e.g. `SID017`). The ledger and the request table join on the card number, because that is the identifier a librarian reads off a card at the desk and types into the issue screen. The card number carries a `UNIQUE` constraint, so the denormalisation cannot produce ambiguity.

Card numbers are allocated by incrementing a counter file under an exclusive lock, exploiting PHP's Perl-style string increment ("`SID016`" becomes "`SID017`"). The original code read and wrote that file without a lock, so two simultaneous registrations could take the same number. Section 7.4 reports what happens at the `SID999` boundary.

5. Circulation logic

Everything that encodes a library rule lives in `includes/library.php`. This section states the rules, then shows how the two operations that change state are made atomic.

5.1 The policy

Constant	Default	Meaning
LOAN_PERIOD_DAYS	14	Days a copy may be kept before a fine begins to accrue
FINE_PER_DAY	5.00	INR charged for each day past the due date
MAX_BOOKS_PER_USER	3	Simultaneous open loans permitted to one member

Table 6. The circulation policy, declared in `includes/config.php`.

A loan is one row in the ledger. Its state is not stored beyond the returned flag: "overdue" is derived from the issue date and the policy every time a page is rendered, so there is no nightly job that can fall behind and no denormalised status to go stale.

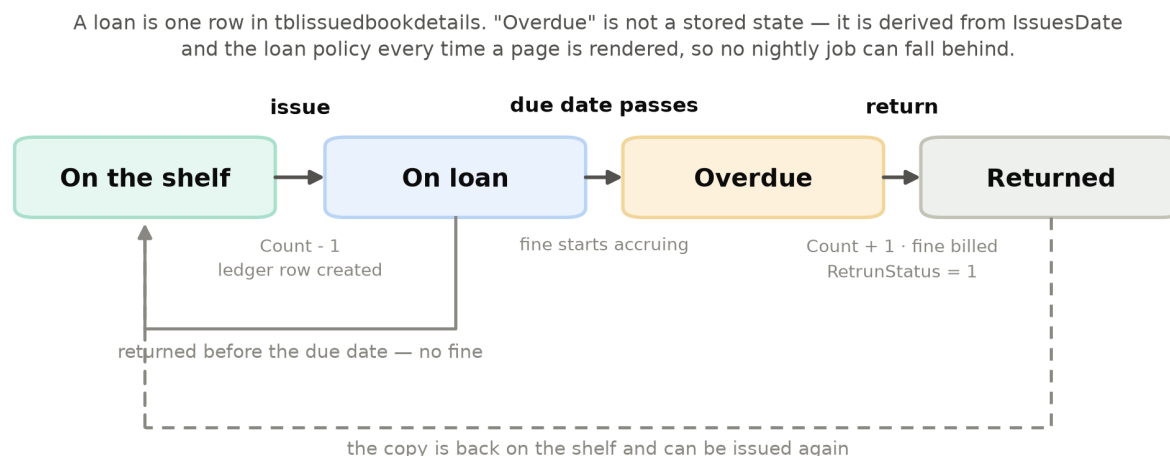


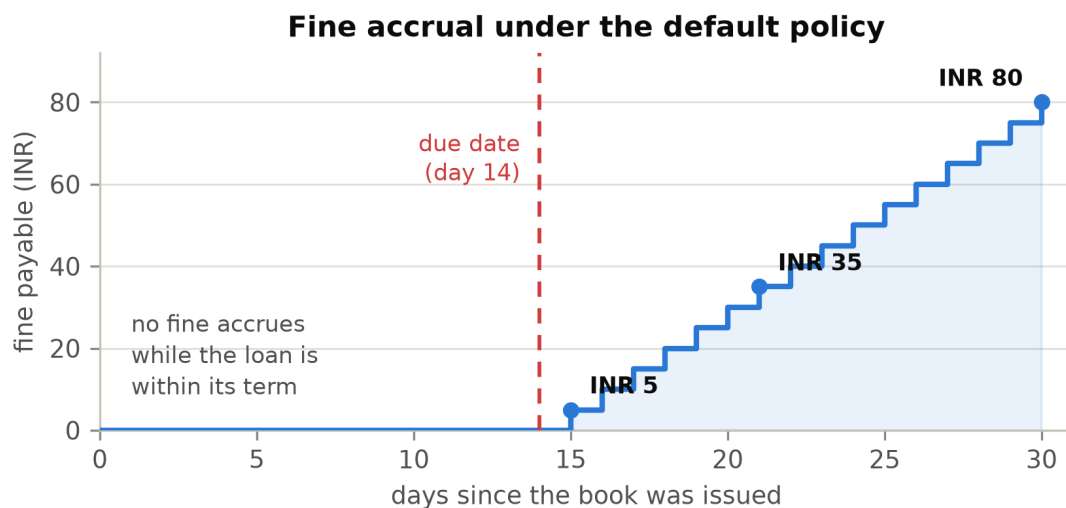
Figure 3. The lifecycle of a single copy. The only stored state is `RetrunStatus`; "overdue" is computed on demand from `IssuesDate` and the loan policy.

5.2 Fine arithmetic

The fine is a step function of the time the copy has been held:

```
fine = max(0, days_since_issue - LOAN_PERIOD_DAYS) × FINE_PER_DAY
```

It is evaluated for the librarian on the return screen and pre-filled, but remains editable. That is deliberate: waivers, damaged copies and disputed dates are ordinary library business, and a system that computes a fine but refuses to let staff override it simply gets worked around. When a loan is closed the fine is frozen at the return date, so a record that is examined months later still shows what was actually charged.



LOAN_PERIOD_DAYS = 14, FINE_PER_DAY = 5.00 | values produced by `lms_calculate_fine()`

Figure 4. Fine accrual against time held. No fine accrues through day 14; each subsequent day adds FINE_PER_DAY. Values are those produced by `lms_calculate_fine()`.

5.3 Issuing a copy

An issue request passes through two gates. The first, `lms_issue_blocker()`, produces the message the librarian reads: unknown member, blocked account, no copy on the shelf, the member already holds that title, or the loan limit is reached. Collecting the checks in one function keeps the page from accumulating half of them.

The second gate is inside the transaction, and exists because the first one is only a snapshot. Two issues submitted at the same moment can both pass it — a time-of-check/time-of-use window that was measured, not hypothesised, and is reported in Section 7.5. The transaction therefore takes a locking read of the member row and re-checks the per-member rules before writing anything.

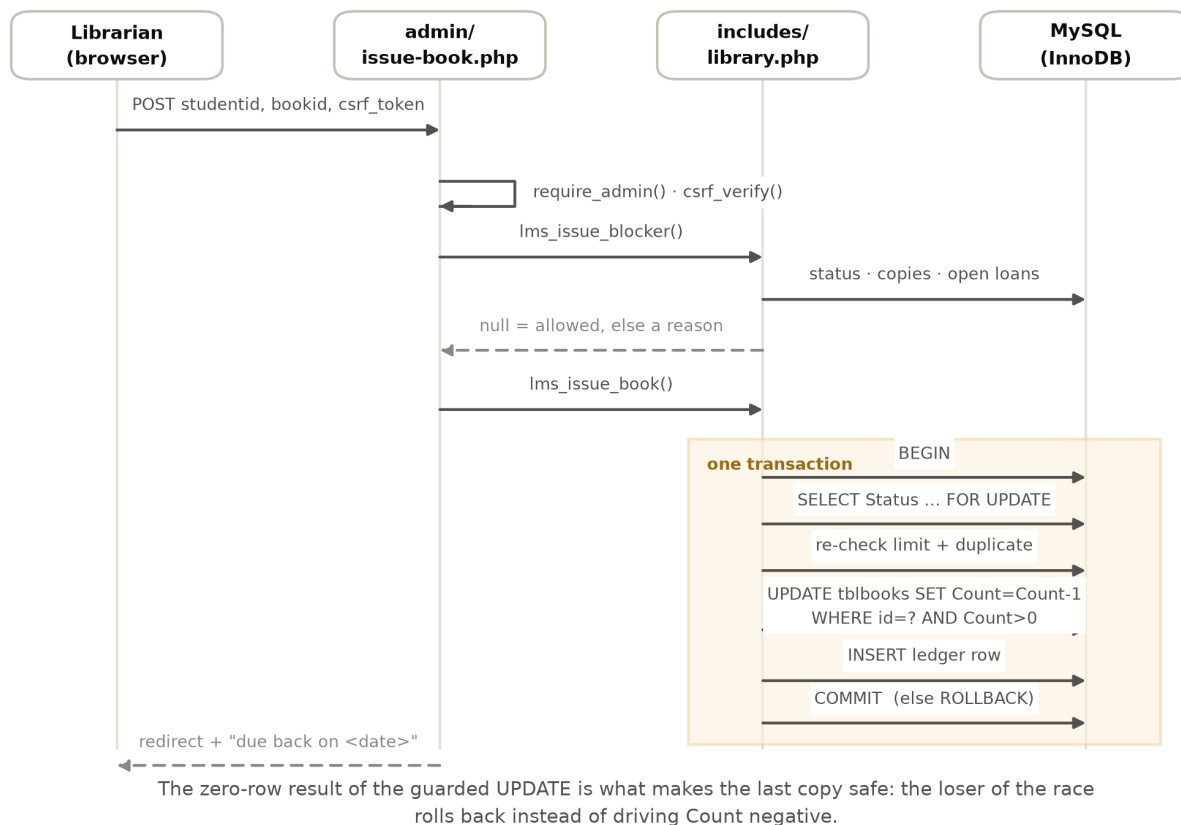


Figure 5. Issuing a copy. The per-member rules are re-checked inside the transaction behind `SELECT ... FOR UPDATE`; the shelf count is protected by a condition inside the `UPDATE` itself.

The two guards differ in kind, and the distinction is the heart of the design. The shelf count is protected by a condition carried inside the write:

```
UPDATE tblbooks SET Count = Count - 1 WHERE id = :id AND Count > 0
```

If two librarians issue the last copy simultaneously, the second statement matches zero rows, the transaction rolls back, and the page reports that the copy has just gone. No lock is required, because the check and the write are one operation. The per-member rules cannot be expressed that way — they are a count across another table — so they need the row lock.

5.4 Receiving a copy

Returning closes the ledger row, restores the copy to the shelf and bills the fine, in one transaction. The ledger row is selected `FOR UPDATE`, so a double submission finds the loan already closed and is refused rather than crediting a second copy to the shelf.

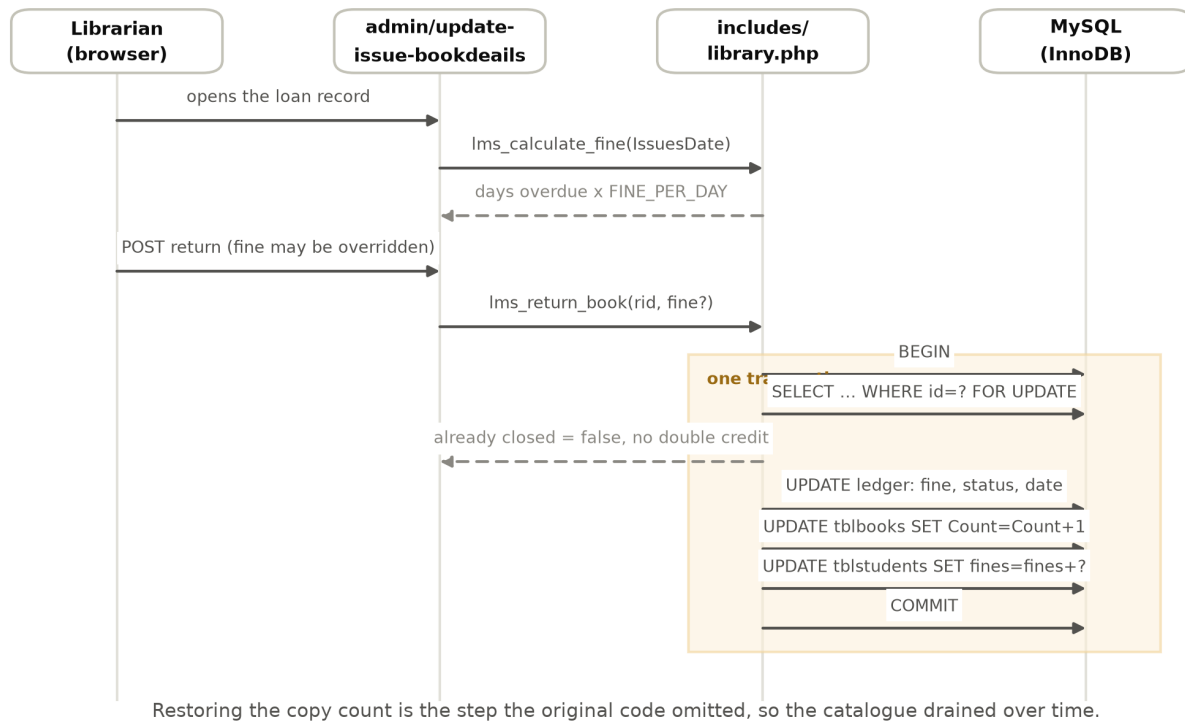


Figure 6. Receiving a copy. Restoring the shelf count is the step the inherited code omitted entirely, which drained the catalogue over time.

6. Security engineering

The inherited application stored unsalted md5 password digests, changed state through GET links, and interpolated query results directly into HTML. This section sets out the threats considered and the control applied to each.

6.1 Threats and controls

Threat	Control applied
Password disclosure from a database leak	bcrypt via <code>password_hash()</code> / <code>password_verify()</code> , replacing unsalted md5
SQL injection	PDO prepared statements everywhere, with emulation disabled so parameters are bound by the server
Cross-site scripting	All dynamic output escaped through a single helper <code>e()</code> (<code>htmlspecialchars</code> , <code>ENT_QUOTES</code> , UTF-8)
Cross-site request forgery	Per-session token in every state-changing form; <code>lms_csrf_verify()</code> rejects a mismatch with HTTP 400
State change by link traversal	Deleting a book, blocking a member and ruling on a request are POST forms; no crawler or link pre-fetcher can trigger them
Privilege escalation	A member-facing page accepted <code>?del=<id></code> and deleted from the catalogue; deletion now exists only behind an admin session (Section 8, D1)
Session fixation	<code>session_regenerate_id(true)</code> on every successful login; cookies are <code>HttpOnly</code> , <code>SameSite=Lax</code> , and <code>Secure</code> under HTTPS
Account enumeration	Login failures return one message whether the e-mail is unknown or the password is wrong
Information disclosure between members	A member's request list shows only their own rows; it previously listed every member's
Credential exposure in version control	Database credentials read from the environment or a gitignored local file
Error-message leakage	<code>error_reporting(0)</code> replaced by logging; database errors are logged, not displayed

Table 7. Threats considered and the control applied to each.

6.2 Migrating passwords without a reset

Replacing md5 with bcrypt in a system that already has accounts poses an obvious problem: the plaintext needed to compute a bcrypt hash is only available for an instant, during a login. The design exploits exactly that instant.

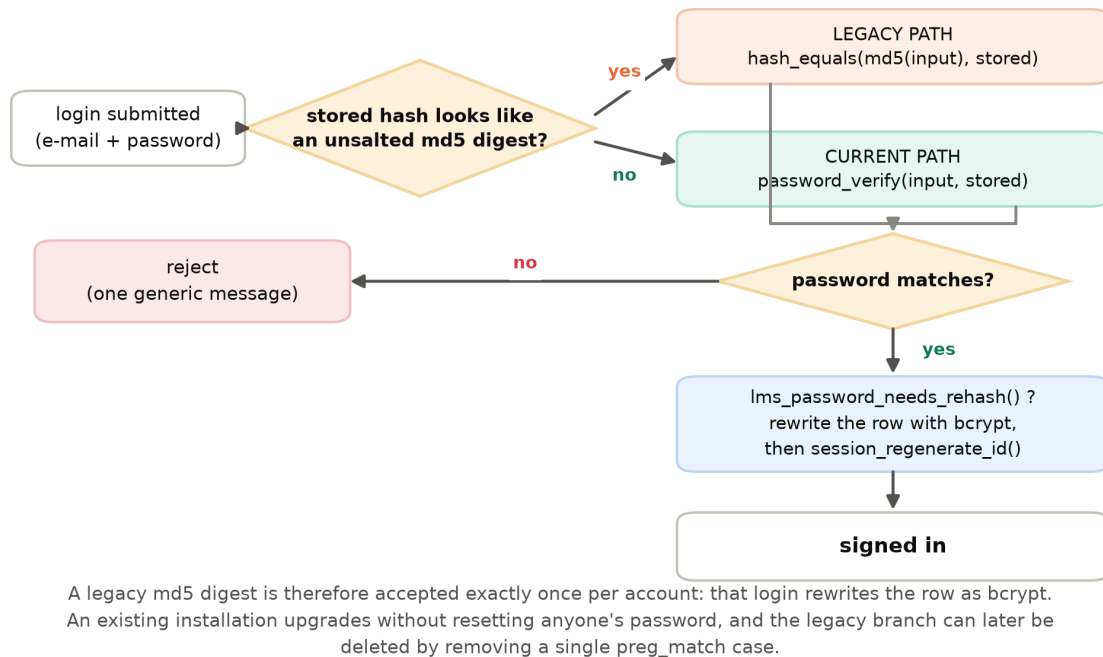


Figure 7. Password verification and transparent migration. A legacy digest is accepted once; that same login rewrites the row as bcrypt.

The password column is widened to 255 characters and may hold either form. A stored value matching the 32-hex-character md5 pattern is compared with a constant-time `hash_equals()`; anything else goes to `password_verify()`. On success, `lms_password_needs_rehash()` decides whether to rewrite the row. An existing installation therefore upgrades itself account by account, with no password reset and no downtime, and the legacy branch can later be deleted by removing a single `preg_match` case.

6.3 Residual risk

- **Login attempts are not rate-limited.** Online password guessing is slowed only by bcrypt's cost factor.
- **The signup e-mail availability endpoint is an enumeration oracle by design.** It is unauthenticated because the signup form needs it; it discloses whether an address is registered. This was accepted as a usability trade-off and is recorded here rather than left implicit.
- **There is one librarian role.** The admin table has no roles or permissions, so any librarian can do anything a librarian can do.
- **The seeded administrator password is public,** being printed in the schema script. It is a demo credential; the report and README both instruct that it be changed at first login.

7. Verification and validation

A constraint shaped this entire section and is stated before any result. The machine on which this work was carried out had no PHP interpreter, no MySQL server, no container runtime and no administrative rights with which to install one. **The PHP source was therefore never executed.** Every claim below rests on analysis of the source, on unit-testable logic, or on a faithful port of the domain layer to a database that was available. Where that is not enough, this report says so rather than implying coverage it does not have.

	Static analysis (3 scripts)	PHP unit tests (17 asserts)	SQLite stress harness (20k ops)	Manual checklist (docs/TESTING)
Table / column names used by the PHP exist	●	·	○	·
Every SQL placeholder is bound exactly once	●	·	·	·
Auth guard on every non-public page	●	·	·	○
CSRF token on every state-changing form	●	·	·	○
Loan policy arithmetic (due dates, fines)	·	●	●	○
Password hashing + legacy md5 upgrade	·	●	·	○
Card-number allocation	·	●	●	·
Schema constraints (PK / UK / FK)	○	·	●	○
Transaction correctness + invariants	·	·	●	○
Concurrency (races on copies and loan limits)	·	·	●	·
PHP runtime behaviour (sessions, rendering)	·	·	·	●
MySQL-specific locking + fixed-point maths	·	·	·	○



primary evidence



partial / supporting

The bottom two rows are the honest gap: no PHP or MySQL was available on the machine where this work was done, so the PHP was never executed and MySQL's own locking was never observed. Those rows rest on the manual checklist, which has not yet been run.

Figure 8. What each verification technique actually covers. Filled markers denote primary evidence, hollow markers supporting evidence. The last two rows are the honest gap.

7.1 Static analysis

Three analyses run over the PHP source with no interpreter required. Each answers a question that would otherwise only surface as a runtime error in front of a user.

Analysis	Question answered	Result
schema_check.py	Does every table and column named in the PHP exist in the schema?	62 SQL statements across 40 files; no mismatch
bind_check.py	Is every declared SQL placeholder bound exactly once, and nothing extra?	49 parameterised statements; all correct
guard_check.py	Does every non-public page carry an auth guard, every POST handler verify CSRF, and every POST form carry a token?	31 pages; complete coverage, no GET-triggered state change remaining

Table 8. Static analyses and their results.

7.2 Unit tests of the database-free logic

tests/run-tests.php contains 17 assertions over the logic that needs no database: due-date computation, the overdue-day count at the day-13, day-14 and day-15 boundaries, the fine formula, a fine frozen at the return date, bcrypt verification, the legacy md5 acceptance path, rehash detection, and card-number allocation

including the SID099-to-SID100 rollover. **These assertions have not been executed**, for the reason given above; they are written and ready for a machine with PHP.

7.3 A stress harness without PHP

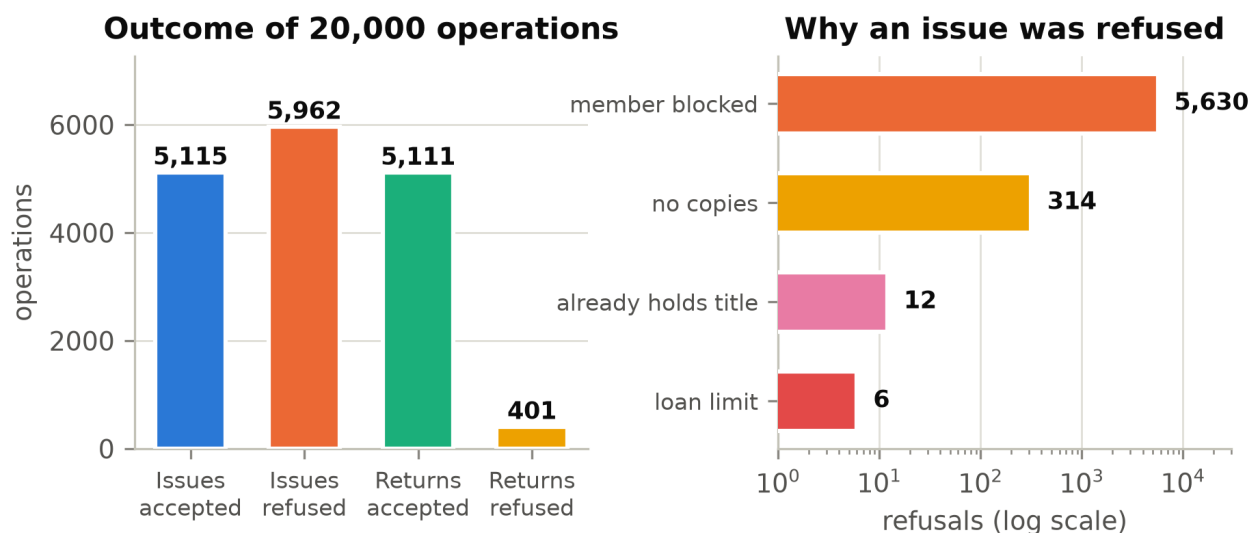
To obtain evidence about the circulation algorithm despite the missing runtime, `includes/library.php` was ported statement-for-statement onto SQLite, which was available through Python's standard library. The schema is not rewritten by hand: a translator reads `database/library.sql` and converts the real DDL, preserving primary, unique and foreign keys, so the constraints under test are the ones that ship.

The harness then drives the system hard while asserting six invariants after every 250 operations:

Invariant	
I1	<code>tblbooks.Count</code> is never negative
I2	For every title: copies on the shelf + open loans = total copies
I3	No member holds more than <code>MAX_BOOKS_PER_USER</code> open loans
I4	No member holds two open loans of the same title
I5	Closed loans carry a return date; open loans do not
I6	<code>tblstudents.fines</code> equals the sum of that member's closed-loan fines

Table 9. The six invariants asserted throughout the stress run.

A run of 20,000 randomised operations completed in 5.5 seconds with no invariant violated. The operation mix and the reasons issues were refused are shown in Figure 9; every refusal branch in `lms_issue_blocker()` was exercised, which is the evidence that the run reached the rules rather than merely the happy path.



Every refusal branch in `lms_issue_blocker()` was exercised; all six invariants held throughout the run.

Figure 9. Outcome of the 20,000-operation randomised run, and the distribution of refusal reasons (logarithmic). One operation in twelve deliberately attempted to close an already-closed loan; none succeeded.

Two scarcity profiles were used, because a single one cannot reach every rule: with one copy per title the "no copies" branch fires before any member can accumulate three loans, while with twelve copies of three titles the loan-limit and duplicate-title branches dominate. Running both is what gives full branch coverage of the policy.

7.4 Card-number allocation

Allocation was exercised over 5,000 sequential draws, all unique. The PHP string-increment behaviour was also emulated at the boundary: SID999 increments to **SIE000**, the carry running into the letters. This looks alarming but is harmless — the sequence continues to produce unique six-character identifiers, and the UNIQUE constraint on the column would reject a collision in any case. It is recorded here because a reader who assumed a numeric counter would expect SID1000.

7.5 Concurrency experiments

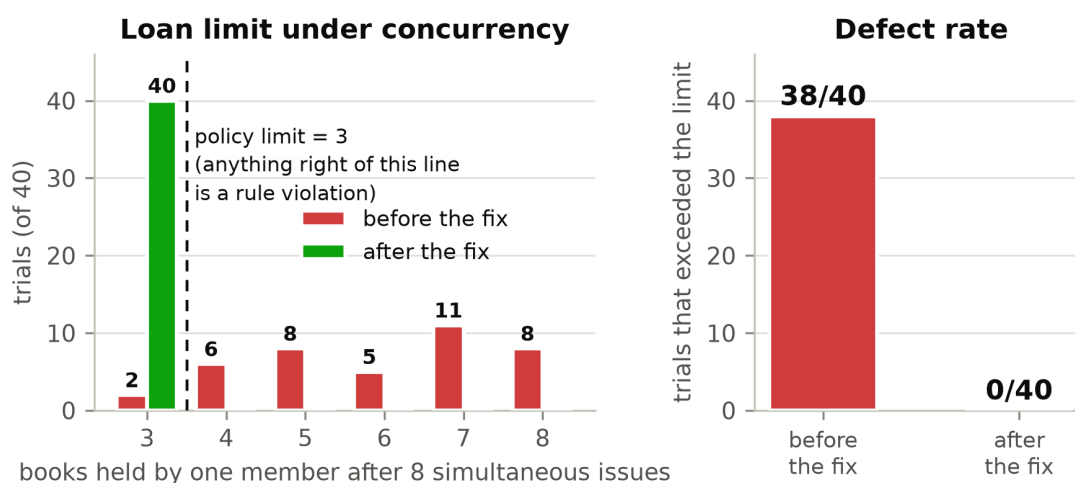
Contending for the last copy

Twelve threads attempted to issue the single remaining copy of one title simultaneously. Exactly 1 succeeded and 11 were refused; the shelf count finished at 0 and the ledger contained 1 row. This is the guarded UPDATE behaving as designed: the losers match zero rows and roll back.

Contending for the loan limit

The same experiment applied to the per-member rules exposed a defect. Eight threads issued eight different titles to one member at once. Because `lms_issue_blocker()` ran before the transaction opened, all eight passed the check: 38 of 40 trials exceeded the limit of 3, with a worst case of 8 books held by a single member.

The fix re-checks the per-member rules inside the transaction, behind a locking read of the member row, which serialises concurrent issues for that member. The experiment was then repeated as a regression test: 0 of 40 trials exceeded the limit, worst case 3.



8 threads issuing 8 different titles to one member at once; 40 trials per configuration. Worst case 8 books before the fix, 3 after.

Figure 10. The loan-limit race before and after the fix. Left: how many books one member ended up holding after 8 simultaneous issues. Right: the proportion of trials that violated the policy.

7.6 Threats to the validity of these results

The harness tests the algorithm and the schema constraints. It is not the deployed system, and four differences matter:

- **SQLite serialises writers across the whole database**, whereas MySQL relies on the row lock taken by `SELECT ... FOR UPDATE`. The results show the algorithm is correct when transactions serialise; they do not prove InnoDB takes the lock as intended.
- **SQLite has no FOR UPDATE**, so `BEGIN IMMEDIATE` stands in for it.

- **DECIMAL became REAL** in translation, so a rounding difference in MySQL's fixed-point arithmetic would not be detected here.
- **No PHP ran.** Sessions, CSRF enforcement at runtime, output escaping as rendered, the AJAX endpoints and every page template are covered only by static analysis and by the manual checklist in docs/TESTING.md, which has not yet been executed.

The single most valuable next step is therefore not more analysis but one run on a machine with PHP and MySQL, following that checklist.

8. Defects found and resolved

Nine defects were identified during the re-engineering. Four affect security (D1, D3, D7, D9), two corrupt the circulation state itself (D2, D4), and three prevented a feature from working or made the application misstate its own data (D5, D6, D8). Each is recorded with how it was found, because the detection method is itself a result.

Defect	Consequence	Found by
D1 A member-facing page accepted <code>?del=<id></code> and deleted that row from the catalogue	Any logged-in member could erase the entire book catalogue	Source review
D2 Returning a book never restored <code>tblbooks.Count</code>	Shelf stock decreased monotonically; the catalogue drained to zero over time	Source review
D3 Per-member rules checked outside the issue transaction	A member could exceed the loan limit under concurrent issues (38/40 trials; worst case 8 books)	Concurrency experiment
D4 Issue decremented the copy count without checking availability, the loan limit or a duplicate loan	Shelf counts could go negative and policy was unenforced	Source review
D5 The admin dashboard prepared the wrong statement variable	"Registered Users" displayed the number of issue records	Source review
D6 The overdue-reminder page filtered on a session key never set for an administrator	The overdue list was always empty, so no reminder could ever be sent	Source review
D7 A member's request page listed every member's requests	Disclosure of what other members were reading	Source review
D8 The reminder link pointed at a page that did not exist	Reminders could not be sent at all	Source review
D9 Card numbers were allocated without a file lock	Two simultaneous registrations could claim the same library card number	Source review

Table 10. Defects found and fixed, with the method that exposed each.

Two further errors were introduced *during* the rework and caught before delivery, which is worth recording because they illustrate what the checks are for. Disabling prepared-statement emulation forbids reusing a named placeholder within one statement; two rewritten queries did exactly that and would have thrown at runtime. The placeholder analysis (Section 7.1) now runs over the whole codebase and reports the condition directly.

9. Results and discussion

9.1 What the evidence supports

- The circulation algorithm maintains all six invariants across 20,000 randomised operations, including 5,115 issues and 5,111 returns, with every policy branch exercised.
- Concurrent contention for the last copy resolves to exactly one winner, with no negative shelf count and no phantom ledger row.
- After the fix, the per-member loan limit holds under simultaneous issues in every trial.

- The schema and the SQL in the application agree completely: no statement references a table or column that does not exist, and no placeholder is left unbound.
- Every non-public page is guarded, every POST handler verifies a CSRF token, and no state change is reachable by a GET request.

9.2 What the evidence does not support

No claim is made that the application runs. It has not been started. A missing semicolon in a template, a mis-typed include path or a version-specific API difference would not have been caught by any check performed here — the structural analysis that was applied to every PHP file verifies tag and brace balance, not executability. The figures and results in Section 7 describe a design that is sound; the deployment remains unverified.

9.3 Reflection on method

The most instructive result was D3, and not because the defect was severe — in a single-desk library, two simultaneous issues to the same member is an unlikely event. It is instructive because the code looked correct, was reviewed twice, and had a comment explaining why it was atomic. Only the experiment exposed the gap between the transaction boundary and the check that mattered. The contrast within the same function — a guard inside the write is safe, a guard before the transaction is not — is the kind of distinction that reads as pedantic in a textbook and becomes concrete when 38 of 40 trials fail.

The second observation concerns the enforced constraint of a machine without PHP. Being unable to run the application forced the domain logic to be separated cleanly enough to be portable to another database, and forced the verification effort into static analysis and invariant-based testing. Both produced findings that a few manual clicks through the user interface would not have.

10. Limitations and future work

Limitation	Consequence	Suggested work
The application has not been executed	Deployment correctness is unverified	Run docs/TESTING.md end to end on a PHP + MySQL host; execute tests/run-tests.php
No reservations or holds	A member who finds every copy out has no way to queue	A reservation table and a claim step at return time
Fines accumulate but are never settled	The balance only ever grows; payments are invisible	A payment ledger with per-transaction records, replacing the running total
Reminders use mail() and are sent by hand	Delivery is unreliable and depends on a librarian remembering	An SMTP transport with a queue, driven by cron
One librarian role	No separation between desk staff and administrators	Roles and a permission check in the admin guard
No rate limiting on login	Online password guessing is slowed only by bcrypt cost	Per-account and per-IP attempt counters with backoff
Bootstrap 3 and jQuery 1.10	Both are long out of support	A front-end refresh, deliberately excluded from this scope
Stress harness is not the deployed engine	MySQL-specific locking and fixed-point arithmetic are unobserved	Re-run the invariant harness against MySQL directly

Table 11. Known limitations, their consequences, and the work each would require.

11. Conclusion

The project set out to turn a coursework application into something that could be read, installed, defended and checked. The schema now exists and is constrained; the circulation rules live in one file and are applied consistently by both portals; the security posture addresses passwords, injection, encoding, forgery and privilege separation; and nine defects, including one that allowed any member to delete the catalogue, have been fixed.

The verification effort is the part worth carrying forward. Working without the application's own runtime forced a discipline that a running system would not have required: state the invariants explicitly, port the logic to something that can be driven at scale, and be precise about the boundary between what has been demonstrated and what has merely been argued. That boundary is drawn in Figure 8 and in Section 7.6, and the single most useful next step is to close it with one session on a machine that has PHP and MySQL installed.

Appendix A. Installing and running

```
# 1. place the project where the web server can serve it
git clone <repository-url> library

# 2. create the schema and seed data
mysql -u root -p < database/library.sql

# 3. point the application at the database (either form)
cp includes/config.local.php.example includes/config.local.php # edit it, or
export LMS_DB_USER=library_app LMS_DB_PASS=secret

# 4. serve it
php -S localhost:8000
```

The member portal is then at / and the librarian panel at /admin/. The seeded librarian account is admin / Test@123; its hash is public in the schema script, so it must be changed at first login. Member accounts are created through the signup form.

Appendix B. Repository layout

Path	Contents
index.php, dashboard.php, ...	Member portal pages
admin/	Librarian panel, including its own AJAX lookups
includes/config.php	Bootstrap: credentials, policy constants, PDO handle
includes/auth.php	Passwords, sessions, CSRF, output escaping
includes/library.php	Circulation rules, fines, transactional issue and return
admin/includes/notify.php	Overdue reminder mail
database/library.sql	Schema and seed data (idempotent)
tests/run-tests.php	Unit assertions over the database-free logic
tests/stress/	Static analyses and the SQLite stress harness
docs/ARCHITECTURE.md	Design decisions and request walkthroughs
docs/TESTING.md	Manual test checklist for the flows needing a database

Appendix C. Reproducing the results in this report

All figures in Sections 7 and 5.2 are generated from measurements, not transcribed. The commands below reproduce them; none requires PHP or MySQL.

```
python3 tests/stress/schema_check.py # Section 7.1
python3 tests/stress/bind_check.py # Section 7.1
python3 tests/stress/guard_check.py # Section 7.1
python3 tests/stress/stress.py # Section 7.3, Figure 9
python3 tests/stress/scenarios.py # Sections 7.4, 7.5
python3 tests/stress/race_loan_limit.py # Section 7.5, Figure 10

php tests/run-tests.php # Section 7.2 (requires PHP)
```

Randomised runs are seeded, so the operation counts quoted in Section 7.3 reproduce exactly. The concurrency experiments are inherently non-deterministic: the defect rate before the fix varies between runs, while the rate after the fix is expected to remain zero.